Long GPU training runs fail often enough to plan for, so set the checkpoint interval from the failure rate

Numbers from a published run on a 504-GPU cluster. Concept, not client work.

On a 504-GPU NVIDIA B200 cluster, training stopped abnormally about every 56 hours

56.2 h

mean time between abnormal terminations

1,294 training hours across 24 runs, with 23 abnormal terminations.

Source: Lablup, arXiv:2605.09370, 2026

Each failure lost about an hour of training, and the restart added half an hour more

0.98 h
mean training time lost per failure

31 min
median time to load a checkpoint on restart

18 s
estimated time to save a checkpoint, 4K phase

Source: Lablup, arXiv:2605.09370, 2026

An interval picked by habit drifts from what the run needs

4K and 32K phases saved about 3x less often than optimal

INTERVAL BY HABIT	INTERVAL FROM MEASUREMENT
Set once at the start	Save time and failure rate measured on this cluster
Kept when sequence length changes	Recomputed when either changes
	Restart time tracked as its own cost

Source: Lablup, arXiv:2605.09370, 2026

With an 18 second save and a 56 hour MTBF, the textbook interval is about 45 minutes

45 min

Young/Daly interval, square root of 2 × save time × MTBF

The 4K phase saved every 2.23 hours. In the 100K phase (optimum 58.1 min), saving every 81.5 minutes put total overhead at 1.82%, near the 1.72% minimum.

Source: Lablup, arXiv:2605.09370, 2026, Table 11

How we would approach it: measure, set the interval, test resume

Illustrative, not a result

01 **Measure**

Time a save and a restart load on your storage. Count abnormal stops per hour.

02 **Set**

Compute the interval. Redo it when sequence length or node count changes.

03 **Test resume**

Our speech fine-tune had no resume after a Slurm requeue until we added one.

When did you last check your checkpoint interval against your failure rate?

Scoping is short, fixed and ends in a written plan.

contact@hopline.co